

Probabilistic Machine Learning

Chapter 1: Introduction

Slides based on K. Murphy, *Probabilistic Machine Learning: An Introduction*

Prepared for course use

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning
- 5 Data
- 6 Discussion

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning
- 5 Data
- 6 Discussion

Mitchell's definition

Definition: Machine Learning (Tom Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

- Many kinds of ML arise from the choice of T (task), P (performance measure), and E (experience/training signal).
- This book treats ML **probabilistically**: every unknown quantity (a future value, a model parameter) is a **random variable** with a probability distribution over its possible values.

Why probabilistic?

1. It is the **optimal** approach to decision making under uncertainty (Section 5.1).
2. Probabilistic modeling is the common language of science and engineering — it unifies ML with control theory, econometrics, information theory, statistical physics, etc.

“Almost all of machine learning can be viewed in probabilistic terms. . . mastery of probabilistic thinking is essential.”

— Shakir Mohamed (DeepMind)

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning
- 5 Data
- 6 Discussion

The supervised learning problem

- Task T : learn a mapping $f : \mathcal{X} \rightarrow \mathcal{Y}$ from inputs to outputs.
- $x = \mathbf{features}$ / covariates / predictors (often $x \in \mathbb{R}^D$).
- $y = \mathbf{label}$ / target / response.
- Experience E : training set $\mathcal{D}_{\text{train}} = \{(x_n, y_n)\}_{n=1}^N$ of N input-output pairs.
- Performance P depends on the type of y : **classification** (y discrete) vs. **regression** (y real-valued).

Classification

Definition: Classification

Output space $\mathcal{Y} = \{1, \dots, C\}$ is a set of C unordered, mutually exclusive **classes**. $C = 2$ is **binary classification** ($y \in \{0, 1\}$ or $\{-1, +1\}$).

Example: Iris flowers

Classify flowers into *Setosa*, *Versicolor*, *Virginica* using 4 numeric features: sepal length/width, petal length/width $\Rightarrow \mathcal{X} = \mathbb{R}^4$. 150 labeled examples, 50 per class — a classic small, clean benchmark.

Figure: the three Iris classes



(a)



(b)



(c)

Figure 1.1: Setosa, Versicolor, and Virginica — distinguishable using 4 simple numeric features.

Figure: image classification as a mapping problem

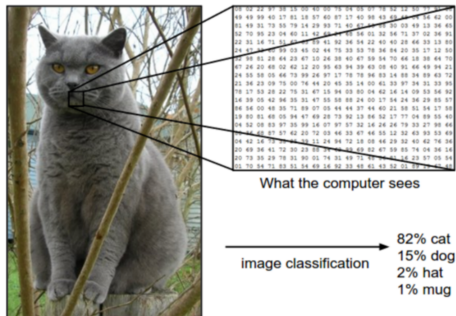


Figure 1.2: A classifier must map raw pixel values (“what the computer sees”) to a label distribution — a far harder input space than the 4-feature Iris case.

The design matrix

- Tabular data with N examples and D features is stored as an $N \times D$ **design matrix**: rows = examples, columns = features.
- Tall-and-skinny ($N \gg D$): “big data”. Short-and-fat ($D \gg N$, e.g. genomics): “wide data”.
- Image classification: $\mathcal{X} = \mathbb{R}^D$ with $D = C_{\text{channels}} \times D_1 \times D_2$ — very high-dimensional; handled by CNNs (Ch. 14).

Exploratory data analysis

- Before modeling: look for patterns / problems (label noise, outliers).
- **Pair plot:** panel (i, j) = scatter of features i, j ; diagonal = marginal density of feature i , color-coded by class.
- High-dim data: first reduce dimensionality (Ch. 20), then visualize in 2D/3D.

Figure: pairwise scatter plot (pair plot) of the Iris data

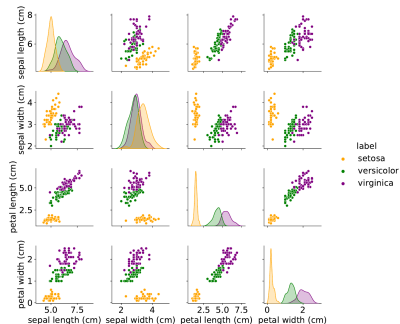


Figure 1.3: Diagonal = marginal density per feature per class; off-diagonals = pairwise scatterplots. Setosa is trivially separable; Versicolor/Virginica overlap more.

A first classifier: decision rules and trees

$$f(x; \theta) = \begin{cases} \text{Setosa} & \text{if } x_{\text{petal length}} < 2.45 \\ \text{Versicolor or Virginica} & \text{otherwise} \end{cases}$$

- A 1-D decision boundary that perfectly separates Setosa.
- Recursively split the remaining region (e.g. threshold on petal width) $\Rightarrow$ **decision tree**.
- Parameters θ : which feature + threshold at each internal node (learning: Section 18.1).

Figure: a depth-2 decision tree on the Iris data

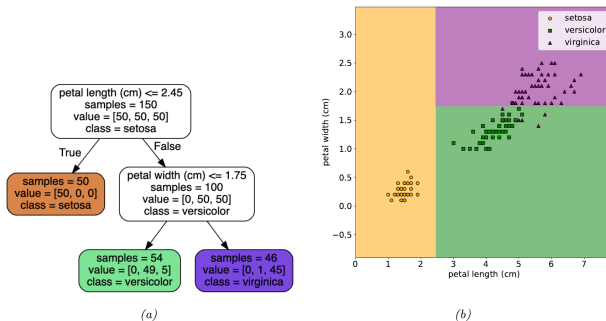


Figure 1.4: (a) Tree splitting first on petal length, then petal width. (b) The resulting axis-aligned decision regions in feature space.

Empirical risk minimization

Key equation

Misclassification rate:
$$L(\theta) \triangleq \frac{1}{N} \sum_{n=1}^N \mathbb{I}[y_n \neq f(x_n; \theta)]$$

- $\mathbb{I}[e] = 1$ if e true, else 0 (indicator function). Assumes all errors equally costly.
- General **loss function** $\ell(y, \hat{y})$ (e.g. asymmetric: eating a poisonous *Virginica* is much worse than misclassifying a *Setosa*).
- **Empirical risk**: $L(\theta) \triangleq \frac{1}{N} \sum_n \ell(y_n, f(x_n; \theta))$. Zero-one loss recovers the misclassification rate.

Model fitting = empirical risk minimization

Key equation

$$\hat{\theta} = \arg \min_{\theta} L(\theta) = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \ell(y_n, f(x_n; \theta))$$

Intuition

The real goal is to minimize **expected** loss on *future* data — i.e. to **generalize**, not just fit the training set (Section 1.2.3).

Uncertainty

- **Epistemic / model uncertainty:** lack of knowledge about the true mapping.
- **Aleatoric / data uncertainty:** intrinsic, irreducible stochasticity.
- Represent uncertainty via a conditional distribution:

$$p(y = c \mid x; \theta) = f_c(x; \theta), \quad 0 \leq f_c \leq 1, \quad \sum_{c=1}^C f_c = 1$$

- Uncertainty matters for decisions: e.g. only eat the flower if $p(\text{Virginica} \mid x)$ is low, or gather more information first (Section 5.1).

The softmax function

To avoid enforcing $\sum_c f_c = 1$ directly, models output unnormalized **logits** $a = f(x; \theta) \in \mathbb{R}^C$ and convert with softmax:

Key equation

$$\text{softmax}(a) \triangleq \left[\frac{e^{a_1}}{\sum_{c'} e^{a_{c'}}}, \dots, \frac{e^{a_C}}{\sum_{c'} e^{a_{c'}}} \right]$$

$$p(y = c \mid x; \theta) = \text{softmax}_c(f(x; \theta))$$

Maps $\mathbb{R}^C \rightarrow [0, 1]^C$ with entries summing to 1 (details: Section 2.5.2).

Logistic regression & linear models

Special case: f affine, $f(x; \theta) = b + w^T x$, with $\theta = (w, b)$ — **logistic regression** (Ch. 10).

- Statistics: w = regression coefficients (β), b = intercept.
- ML / EE: w = weights, b = bias.
- Absorb bias: $\tilde{w} = [b, w_1, \dots, w_D]$, $\tilde{x} = [1, x_1, \dots, x_D]$ so $\tilde{w}^T \tilde{x} = b + w^T x$ — turns affine into linear: $f(x; w) = w^T x$.

Maximum likelihood estimation

Use negative log probability as the loss:

$$\ell(y, f(x; \theta)) = -\log p(y \mid f(x; \theta))$$

Key equation

Negative log-likelihood (NLL):

$$\text{NLL}(\theta) = -\frac{1}{N} \sum_{n=1}^N \log p(y_n \mid f(x_n; \theta))$$

$$\hat{\theta}_{\text{mle}} = \arg \min_{\theta} \text{NLL}(\theta)$$

Intuition

A good model assigns *high* probability to the true label $\Rightarrow$ low NLL. (Justified fully in Section 5.1.6.1.)

Regression

Definition: Regression

Predict a real-valued output $y \in \mathbb{R}$ (e.g. toxicity level, plant height).

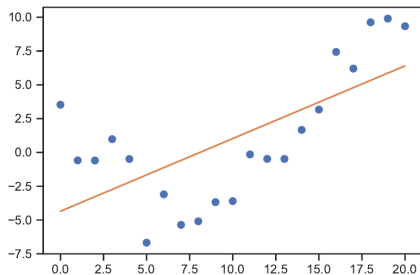
Key equation

Quadratic (ℓ_2) loss: $\ell_2(y, \hat{y}) = (y - \hat{y})^2$

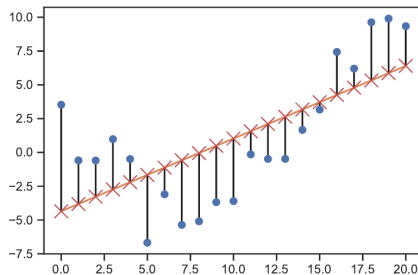
Mean squared error: $\text{MSE}(\theta) = \frac{1}{N} \sum_{n=1}^N (y_n - f(x_n; \theta))^2$

Large residuals penalized more (use ℓ_1 loss instead if outliers are a concern — Section 11.6).

Figure: fitting a line by least squares



(a)



(b)

Figure 1.5: (a) A fitted line. (b) The residuals (vertical lines) between observed points and predictions — least squares minimizes their squared sum.

Regression as Gaussian likelihood

Model output uncertainty with a Gaussian: $p(y_n | x_n; \theta) = \mathcal{N}(y_n | f(x_n; \theta), \sigma^2)$, where

$$\mathcal{N}(y | \mu, \sigma^2) \triangleq \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y - \mu)^2\right)$$

Key equation

With σ^2 fixed: $\text{NLL}(\theta) = \frac{1}{2\sigma^2} \text{MSE}(\theta) + \text{const}$

Intuition

MLE under a Gaussian likelihood $\equiv$ least-squares regression.

Linear & multiple linear regression

$$f(x; \theta) = b + wx \quad (1D) \quad \hat{\theta} = \arg \min_{\theta} \text{MSE}(\theta)$$

$$f(x; \theta) = b + w_1x_1 + \cdots + w_Dx_D = b + w^T x \quad (\text{multiple linear regression})$$

- Least-squares solution minimizes the sum of squared vertical residuals.
- Example: predicting room temperature from 2D location, $f(x) = b + w_1x_1 + w_2x_2$ (details: Section 11.2.2.1).

Figure: linear vs. polynomial regression in 2d

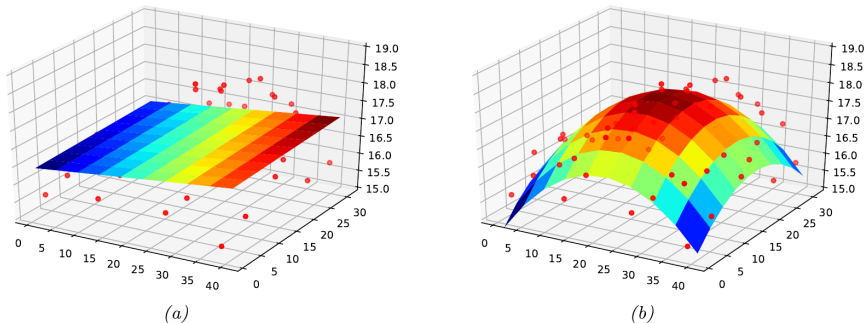


Figure 1.6: (a) A fitted plane (linear in 2 inputs). (b) A fitted quadratic surface — more flexible, better fit to the curved temperature data.

Polynomial regression & feature engineering

$$f(x; w) = w^T \phi(x), \quad \phi(x) = [1, x, x^2, \dots, x^D]$$

- Still **linear in the parameters** $w \Rightarrow$ MSE has a unique global optimum (Section 11.2.2.1).
- Manually designing $\phi(x)$ = **feature engineering**.
- Can add cross terms ($x_1 x_2$) to capture interactions; 2D quadratic expansion improves the temperature-surface fit.

Degrees of freedom and overfitting (preview)

- As polynomial degree $D \rightarrow N - 1$: one parameter per data point $\Rightarrow$ perfect interpolation, **MSE** $\rightarrow 0$ on training data.
- But the fit becomes “wiggly” and predicts poorly on new inputs.
- Full treatment next in Section 1.2.3.

Figure: overfitting as polynomial degree increases

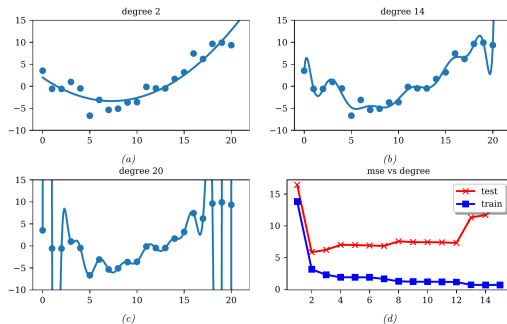


Figure 1.7: (a-c) Degree 2/14/20 fits to the same 21 points. (d) Train MSE keeps falling, but test MSE bottoms out then rises — classic overfitting.

From polynomials to deep neural networks

Learn the feature transform itself: let $\phi(x; V)$ have its own parameters V :

$$f(x; w, V) = w^T \phi(x; V)$$

Recursively compose L simple functions:

Key equation

$$f(x; \theta) = f_L(f_{L-1}(\cdots f_1(x) \cdots))$$

Final layer linear: $f_L(x) = w_L^T x$, so $f(x; \theta) = w_L^T f_{1:L-1}(x)$ — a **learned** feature extractor. This is the core idea of **deep neural networks** (DNNs): CNNs (images, Ch. 14), RNNs (sequences, Ch. 15).

Empirical risk vs. population risk

$$L(\theta; \mathcal{D}_{\text{train}}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} \ell(y, f(x; \theta))$$

Definition: Population risk

$$L(\theta; p^*) \triangleq \mathbb{E}_{p^*(x,y)}[\ell(y, f(x; \theta))]$$

Generalization gap: $L(\theta; p^*) - L(\theta; \mathcal{D}_{\text{train}})$. Large gap (low train risk, high population risk)
 $\Rightarrow$ **overfitting**.

Train / validation / test

- p^* unknown $\Rightarrow$ approximate population risk with **test risk** $L(\theta; \mathcal{D}_{\text{test}})$.
- Polynomial degree D : training error $\rightarrow 0$ as D grows; test error is **U-shaped** — underfitting (D small), overfitting (D large), “just right” in between.
- Never use the training set to choose model complexity (it always favors the most complex model).
- In practice: split into **train** / **validation** (model selection) / **test** (final, unbiased performance estimate) – Section 4.5.4.

No free lunch theorem

“All models are wrong, but some models are useful.” — George Box

- No single model is best for *all* problems (Wolpert, 1996) — performance depends on the match between a model's **inductive bias** and the problem.
- Best practice: use domain knowledge and/or model selection (cross-validation Section 4.5.4, Bayesian methods Sections 5.2.2, 5.2.6) — keep many tools in the toolbox.

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning
- 5 Data
- 6 Discussion

Unsupervised learning

Definition: Unsupervised learning

Given only inputs $\mathcal{D}_{\text{train}} = \{x_n\}_{n=1}^N$ (no outputs), fit an **unconditional** model $p(x)$ that can generate new data — vs. supervised learning's **conditional** model $p(y | x)$.

Motivations:

- Avoids costly large labeled datasets.
- Avoids forcing the world into arbitrary label categories.
- Forces the model to explain the full high-dimensional input $\Rightarrow$ richer world models.
“...there's only one place you can get that much information: from the input itself.” — Geoffrey Hinton, 1996

Clustering

Definition: Clustering

Partition inputs into regions of “similar” points.

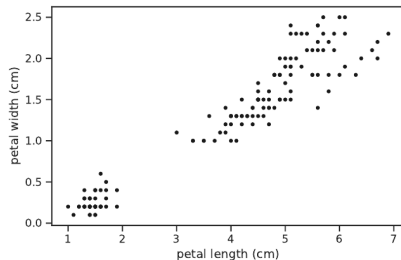
Example: 2D Iris data

Visually ≥ 2 clusters; requiring compactness suggests splitting further into 3.

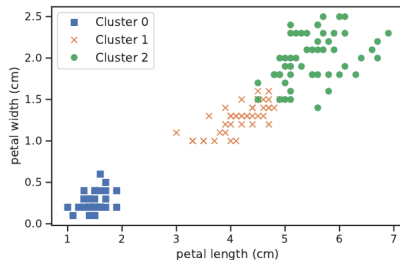
Intuition

There is no single “correct” number of clusters — it trades off model complexity against fit to data (Section 21.3.7).

Figure: clustering the Iris data (unsupervised)



(a)



(b)

Figure 1.8: (a) Raw, unlabeled scatterplot. (b) K -means with $K=3$ clusters recovers structure closely matching the true (hidden) species labels.

Discovering latent factors

Assume each high-dim output $x_n \in \mathbb{R}^D$ is generated from low-dim latent factors $z_n \in \mathbb{R}^K$: $z_n \rightarrow x_n$.

$$p(x_n | z_n; \theta) = \mathcal{N}(x_n | Wz_n + \mu, \Sigma) \quad \text{factor analysis (FA)}$$

- Like linear regression, but z_n is *unobserved*.
- Special case $\Sigma = \sigma^2 I$: **probabilistic PCA** (Section 20.1).
- Nonlinear extension: $p(x_n | z_n; \theta) = \mathcal{N}(x_n | f(z_n; \theta), \sigma^2 I)$ with f a DNN $\Rightarrow$ **variational autoencoder** (Section 20.3.5).

Figure: discovering a low-dimensional latent manifold

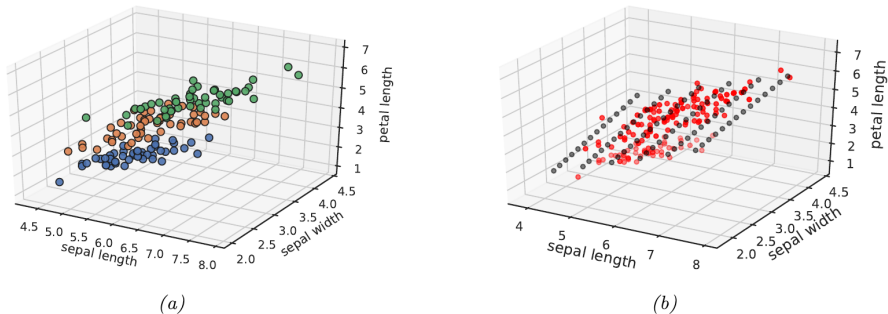


Figure 1.9: (a) Iris data in 3d. (b) PCA fits a 2d linear subspace ($\hat{x} = Wz + \mu$); black dots are points generated from the fitted manifold.

Self-supervised learning

Definition: Self-supervised learning

Create proxy supervised tasks from unlabeled data, e.g. predict color from grayscale, or predict masked words from context: $\hat{x}_1 = f(x_2; \theta)$.

- Learns useful features without needing the “true” latent factors z .
- Features transferred to downstream supervised tasks (Section 19.2).

Evaluating unsupervised learning

Hard to evaluate — no ground truth. Two common strategies:

1. **Density estimation**: negative log-likelihood on held-out data

$$L(\theta; \mathcal{D}) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \log p(x | \theta)$$

A good model is not “surprised” by real samples; can be difficult in high dimensions and can be gamed by memorization.

2. **Downstream task performance**: does the learned representation improve **sample efficiency** on a supervised task?

Often the deeper goal is **interpretability** / scientific understanding, not predictive performance.

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning**
- 5 Data
- 6 Discussion

Reinforcement learning

Definition: Reinforcement learning (RL)

An agent learns a **policy** $a = \pi(x)$ mapping states to actions by interacting with an environment and receiving occasional **reward** signals.

Example:

Atari Space Invaders: x = game image, a = move/fire.

Humanoid robot walking: x = joint angles, a = motor torques.

Figure: the three types of ML, as a cake

■ "Pure" Reinforcement Learning (cherry)

- ▶ The machine predicts a scalar reward given once in a while.

▶ **A few bits for some samples**

■ Supervised Learning (icing)

- ▶ The machine predicts a category or a few numbers for each input
- ▶ Predicting human-supplied data

▶ **10→10,000 bits per sample**

■ Unsupervised/Predictive Learning (cake)

- ▶ The machine predicts any part of its input for any observed part.
- ▶ Predicts future frames in videos

▶ **Millions of bits per sample**



Figure 1.11: Yann LeCun's cake analogy — unsupervised/predictive learning (the cake) provides by far the most bits of learning signal per sample; supervised learning is the icing; RL is the cherry on top.

RL vs. supervised learning

- SL: a *teacher* tells you the right output at every step.
- RL: a *critic* gives only an occasional thumbs-up/down — the **credit assignment** problem (which of many actions caused the reward?).
- Reward may be very sparse (e.g. win/lose at the end of a chess game).
- Mitigate with expert demonstrations (supervised) or unlabeled interaction data (unsupervised) to improve sample efficiency.

“If intelligence was a cake, unsupervised learning would be the [sponge], supervised learning would be the icing, and reinforcement learning would be the cherry.” — Yann LeCun, NeurIPS 2016

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning
- 5 Data**
- 6 Discussion

Common image datasets

- **MNIST**: 60k train / 10k test, 28×28 grayscale digits, 10 classes. “The drosophila of ML” (Hinton) — now considered too easy.
- **EMNIST**: adds upper/lowercase letters, 62 classes, harder.
- **Fashion-MNIST**: same size/shape as MNIST, clothing images, harder.
- **CIFAR-10/100**: 60k $32 \times 32 \times 3$ color images, everyday objects.

Figure: the MNIST handwritten-digit dataset

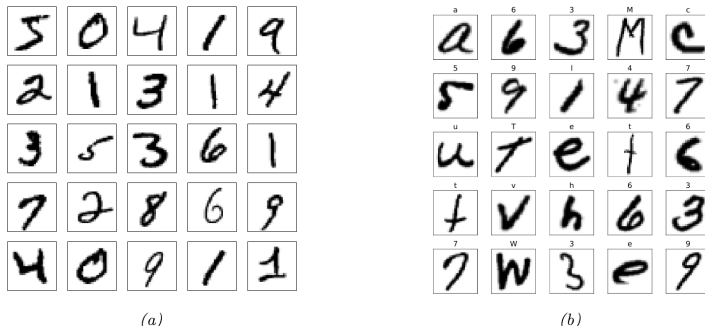


Figure 1.12: (a) MNIST: 60k training / 10k test, 28×28 grayscale digits 0–9. (b) EMNIST: a harder 62-class extension (a–z, A–Z, 0–9).

- $\sim 14\text{M}$ images, $256 \times 256 \times 3$, 20,000 classes.
- **ILSVRC** (2010–2018): 1.3M images, 1000 classes, top-5 error metric.
- 2015: CNNs first surpassed human (Karpathy) top-5 accuracy.
- Now near-saturated, but relative performance on ImageNet still predicts performance on other vision tasks.

Common text datasets & tasks

- **Text classification:** e.g. IMDB movie reviews (25k train / 25k test, binary sentiment).
- **Machine translation:** aligned (x, y) sentence pairs (e.g. WMT English–German).
- **Seq2seq:** general sequence-to-sequence mapping (summarization, QA – e.g. SQuAD).
- **Language modeling:** unconditional generative model of text, $p(x_1, \dots, x_T)$ — a form of unsupervised learning.

Preprocessing discrete inputs

Definition: One-hot (dummy) encoding

K -valued categorical x : $\text{one-hot}(x) = [\mathbb{I}[x = 1], \dots, \mathbb{I}[x = K]]$.

Key equation

Feature crosses capture interaction effects a linear model on marginal one-hots cannot:

$$f(x; w) = w_0 + \sum_k w_k \mathbb{I}[x_1 = k] + \sum_j w_j \mathbb{I}[x_2 = j] + \sum_{k,j} w_{kj} \mathbb{I}[x_1 = k, x_2 = j]$$

E.g. “trucks from the USA” may be less efficient than the type/origin main effects alone predict.

Preprocessing text: bag of words

Key equation

$$\tilde{x}_{nv} = \sum_{t=1}^T \mathbb{I}[x_{nt} = v] \quad (\text{count of word } v \text{ in document } n)$$

- Ignores word order; documents $\rightarrow$ vectors in $\mathbb{R}^D$ (**vector space model**).
- Pre-processing: lower-casing, stop-word removal, stemming.
- Stored as a $D \times N$ **term-frequency matrix**.

TF-IDF and word embeddings

Key equation

$$\text{IDF}_i \triangleq \log \frac{N}{1 + \text{DF}_i}, \quad \text{TFIDF}_{ij} = \log(\text{TF}_{ij} + 1) \times \text{IDF}_i$$

Downweights uninformative, high-frequency words.

Word embeddings: map one-hot $x_{nt} \in \{0, 1\}^V$ to dense $e_{nt} = Ex_{nt} \in \mathbb{R}^K$, $E \in \mathbb{R}^{K \times V}$ learned so similar words are nearby (Section 20.5). Document embedding: $e_n = \sum_t e_{nt} = E\tilde{x}_n$, used e.g. in $p(y = c \mid x_n, \theta) = \text{softmax}_c(WE\tilde{x}_n)$.

Out-of-vocabulary words

- Novel words at test time (**OOV problem**) — vocabulary is an open class (e.g. proper nouns).
- Heuristic: map to a special UNK token (loses information).
- Better: **subword units** / **wordpieces** via **byte-pair encoding** — exploit substructure within words.

Handling missing data

Let $M_{nd} = 1$ if feature d of example n is missing.

- **MCAR**: $p(M \mid X_v, X_h, Y) = p(M)$ — missingness independent of everything.
- **MAR**: $p(M \mid X_v, X_h, Y) = p(M \mid X_v, Y)$ — depends only on *observed* features.
- **NMAR**: depends on the missing values themselves (e.g. a sensitive survey question) — must model the missingness mechanism.

This book assumes **MAR**. Simple fix: **mean imputation**; better: fit a generative model (Ch. 20) to impute.

Outline

- 1 What is machine learning?
- 2 Supervised learning
 - Classification
 - Regression
 - Overfitting and generalization
 - No free lunch theorem
- 3 Unsupervised learning
 - Clustering
 - Latent factors of variation
 - Self-supervised learning
 - Evaluating unsupervised learning
- 4 Reinforcement learning
- 5 Data
- 6 Discussion

ML and neighboring fields

- **Predictive analytics** $\approx$ supervised learning, business-focused.
- **Data mining** $\approx$ supervised + unsupervised ML on structured/database data.
- **Data science**: ML + statistics + data integration/visualization + domain expertise.
- Deeply related to **statistics** (“[without computing,] many other data fields would have been part of statistics” – J. Friedman) and to **AI** (ML now the dominant route to building AI systems).

Structure of this book

- Part I: statistical **foundations**.
- Parts II–IV: **supervised** learning (linear models, deep nets, nonparametric methods).
- Part V: **unsupervised** & beyond.
- Companion notebooks: probml.github.io/book1. Sequel volume *Advanced Topics* covers RL and more.

Caveats: reward hacking & alignment

- Hard to fully specify a loss function that captures all our true preferences $\Rightarrow$ **reward hacking**: the optimizer exploits gaps in the specified objective.
- This is an instance of the **alignment problem**: discrepancy between what we ask for and what we actually want.
- **Inverse reinforcement learning (IRL)**: infer the reward from observed human behavior instead of hand-specifying it (Russell's "assistance games").

AGI vs. augmented intelligence

- **AGI** view: autonomous systems that make decisions without a human in the loop.
- **Augmented intelligence (IA)**: AI as “smart tools” (autopilot, adaptive cruise control) that keep a human in the loop.
- As tools grow more capable, the line between the two blurs.

Summary

- ML = learn from experience E to improve at task T per measure P ; this book takes a **probabilistic** view throughout.
- Three main paradigms: **supervised** (classification/regression), **unsupervised** (density estimation, clustering, latent factors), **reinforcement learning**.
- Core recurring machinery: loss functions, empirical risk minimization, MLE/NLL, train/val/test splits, generalization.
- Data representation matters: design matrices, one-hot encodings, bag-of-words / TF-IDF / embeddings, handling missing data.